

PE-EC603D — Information Theory and Coding

Semester VI • 3L:0T:0P • 3 Credits

Module 3 Exam Notes: Coding Techniques & Cyclic/Convolutional Codes

CGEC | B.Tech ECE (2023–27) | *Biraj Sarkar*

Contents

1	Huffman Coding	2
1.1	Definition and Concept	2
1.2	The Huffman Coding Algorithm	2
2	Cyclic Codes	3
2.1	Definition of Cyclic Codes	4
2.2	Generator Polynomial $g(x)$	4
2.3	Systematic Encoding of Cyclic Codes	4
2.4	Hardware Encoder using Feedback Shift Registers (FSR)	5
3	Convolutional Codes	6
3.1	Basic Principles and Structure	6
3.2	Representations of Convolutional Codes	6
3.3	Viterbi Decoding Algorithm	7
4	Arithmetic Coding	7
4.1	Core Concept	7
4.2	Encoding Procedure	7

1. Huffman Coding

Huffman coding is an optimal source coding technique used for lossy/lossless data compression. It belongs to the class of prefix-free codes, which are uniquely detectable.

1.1 Definition and Concept

Huffman Coding: A bottom-up, variable-length source coding algorithm that assigns binary codes to symbols based on their probabilities. High-probability symbols are assigned shorter codewords, and low-probability symbols are assigned longer codewords, minimizing average codeword length.

1.2 The Huffman Coding Algorithm

- Step 1:** Sort the source symbols in descending order of their probabilities.
- Step 2:** Combine the two lowest-probability symbols into a single node (parent node) with a probability equal to the sum of their probabilities.
- Step 3:** Assign binary '0' to one branch of the pair (usually the upper/higher probability branch) and '1' to the other branch.
- Step 4:** Place the combined node back into the list and re-sort all items in descending order.
- Step 5:** Repeat Steps 2 to 4 recursively until only one root node (probability = 1.0) remains.
- Step 6:** The codeword for each symbol is read by tracing the path from the root of the tree down to the symbol leaf node.

Worked Example: Huffman Coding

Problem: A discrete source emits 5 symbols $\{s_1, s_2, s_3, s_4, s_5\}$ with probabilities $P = \{0.4, 0.2, 0.2, 0.1, 0.1\}$.

- a. Construct the Huffman code tree and determine the codewords.
- b. Calculate the average codeword length L and coding efficiency η .

Solution:

(a) Tree Construction and Codewords: Tracing from the lowest probabilities up (bottom-up merging):

- Merge s_4 (0.1) and s_5 (0.1) to create node N_1 with probability 0.2. Assign 0 to s_4 and 1 to s_5 .
- We now have nodes with probabilities $\{0.4, 0.2, 0.2, 0.2\}$. Merge the two lowest, say s_3 (0.2) and N_1 (0.2) to create N_2 with probability 0.4. Assign 0 to s_3 and 1 to N_1 .
- We now have $\{0.4, 0.4, 0.2\}$. Merge s_2 (0.2) and N_2 (0.4) to create N_3 with probability 0.6. Assign 0 to s_2 and 1 to N_2 .
- Merge s_1 (0.4) and N_3 (0.6) to create the root (1.0). Assign 0 to s_1 and 1 to N_3 .

Symbol	Probability (P_i)	Codeword Path	Length (l_i)
s_1	0.4	0	1
s_2	0.2	10	2
s_3	0.2	110	3
s_4	0.1	1110	4
s_5	0.1	1111	4

(b) Calculations: Source Entropy $H(X)$:

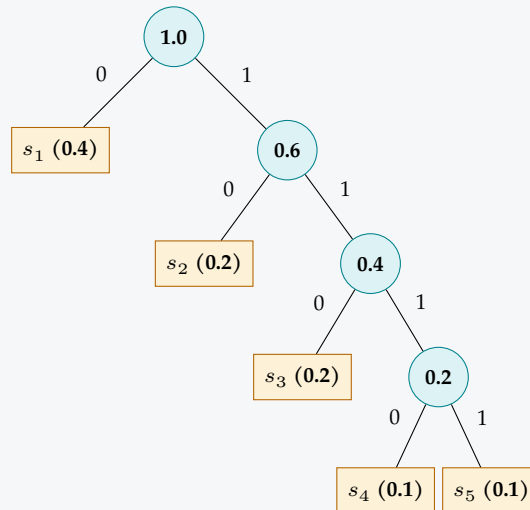
$$\begin{aligned}
 H(X) &= -[0.4 \log_2 0.4 + 2 \times (0.2 \log_2 0.2) + 2 \times (0.1 \log_2 0.1)] \\
 &= -[0.4(-1.3219) + 0.4(-2.3219) + 0.2(-3.3219)] \\
 &= -[-0.5288 - 0.9288 - 0.6644] = 2.122 \text{ bits/symbol}
 \end{aligned}$$

Average Codeword Length L :

$$\begin{aligned}
 L &= \sum P_i l_i = (0.4 \times 1) + (0.2 \times 2) + (0.2 \times 3) + (0.1 \times 4) + (0.1 \times 4) \\
 &= 0.4 + 0.4 + 0.6 + 0.4 + 0.4 = 2.2 \text{ bits/symbol}
 \end{aligned}$$

$$\text{Efficiency } \eta = \frac{H(X)}{L} = \frac{2.122}{2.2} \times 100\% = 96.45\%$$

Huffman Coding Tree Diagram



2. Cyclic Codes

Channel coding techniques inject structured redundancy into data packets to allow detection and correction of channel-induced errors.

2.1 Definition of Cyclic Codes

Cyclic Code: A linear block code (n, k) with the algebraic property that any cyclic shift of a valid codeword is also a valid codeword.

If $C = (c_0, c_1, \dots, c_{n-1})$ is a codeword, then $C^{(1)} = (c_{n-1}, c_0, \dots, c_{n-2})$ is also a codeword.

In polynomial representation, a codeword is represented as:

$$c(x) = c_0 + c_1x + c_2x^2 + \dots + c_{n-1}x^{n-1} \quad (1)$$

The cyclic shift property means that if $c(x)$ is a codeword, then the remainder of $xc(x)$ divided by $x^n + 1$ is also a codeword.

2.2 Generator Polynomial $g(x)$

Every cyclic code is uniquely defined by a **generator polynomial** $g(x)$ of degree $n - k$.

- $g(x)$ must be a factor of the polynomial $x^n + 1$ over GF(2) (Galois Field of 2, modulo-2 arithmetic).
- A polynomial $c(x)$ of degree $\leq n - 1$ is a valid codeword if and only if it is a multiple of $g(x)$:

$$c(x) = m(x) \cdot g(x) \quad (2)$$

Where $m(x)$ is the message polynomial of degree $\leq k - 1$.

2.3 Systematic Encoding of Cyclic Codes

In systematic codes, the first k bits represent the message directly, and the remaining $n - k$ bits are parity check bits.

Systematic Cyclic Coding Steps

For message polynomial $m(x)$ and generator polynomial $g(x)$ of degree $n - k$:

Step 1: Multiply the message polynomial by x^{n-k} :

$$u(x) = x^{n-k}m(x)$$

Step 2: Divide $u(x)$ by $g(x)$ to obtain the remainder polynomial $r(x)$:

$$\frac{x^{n-k}m(x)}{g(x)} = q(x) + \frac{r(x)}{g(x)} \implies r(x) = \text{rem} \left[\frac{x^{n-k}m(x)}{g(x)} \right]$$

Step 3: Form the codeword polynomial $c(x)$ by adding the remainder $r(x)$ to $u(x)$ (addition is XOR in GF(2)):

$$c(x) = x^{n-k}m(x) + r(x) \quad (3)$$

Worked Example: (7, 4) Cyclic Systematic Code

Problem: For a (7, 4) cyclic code, the generator polynomial is $g(x) = x^3 + x + 1$. Encode the message vector $m = [1, 0, 1, 1]$ systematically.

Solution: Identify parameters: $n = 7, k = 4, n - k = 3$. Convert message vector to polynomial:

$$m(x) = 1 \cdot x^0 + 0 \cdot x^1 + 1 \cdot x^2 + 1 \cdot x^3 = 1 + x^2 + x^3$$

Multiply $m(x)$ by $x^{n-k} = x^3$:

$$x^3 m(x) = x^3(1 + x^2 + x^3) = x^3 + x^5 + x^6$$

Divide $x^3 m(x)$ by $g(x) = x^3 + x + 1$ using modulo-2 long division:

Divide: $x^6 + x^5 + x^3$ by $x^3 + x + 1$

$$x^6 + x^5 + x^3 = x^3(x^3 + x + 1) + x^5 + x^4 \quad (\text{since } x^6 + x^4 + x^3 \text{ subtracted leaves } x^5 + x^4)$$

Continuing long division gives:

$$\begin{array}{r} x^3 + x + 1 \overline{) x^6 + x^5 + 0x^4 + x^3} \\ \underline{x^6 + 0x^5 + x^4 + x^3} \quad (\text{multiply by } x^3) \\ x^5 + x^4 + 0x^3 \\ \underline{x^5 + 0x^4 + x^3 + x^2} \quad (\text{multiply by } x^2) \\ x^4 + x^3 + x^2 \\ \underline{x^4 + 0x^3 + x^2 + x} \quad (\text{multiply by } x) \\ x^3 + 0x^2 + x \\ \underline{x^3 + 0x^2 + x + 1} \quad (\text{multiply by } 1) \\ 1 \quad (\text{Remainder}) \end{array}$$

Thus, the remainder is $r(x) = 1$. Construct the codeword:

$$c(x) = x^3 m(x) + r(x) = (x^6 + x^5 + x^3) + 1 = 1 + x^3 + x^5 + x^6$$

Convert polynomial to codeword vector:

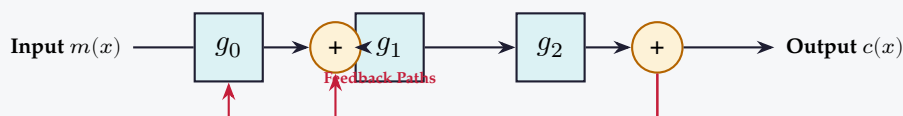
$$C = [1, 0, 0, 1, 0, 1, 1]$$

Where the first 3 bits $[1, 0, 0]$ represent the parity check bits, and the last 4 bits $[1, 0, 1, 1]$ represent the message.

2.4 Hardware Encoder using Feedback Shift Registers (FSR)

Cyclic codes are popular because their encoding and syndrome circuits can be implemented efficiently in hardware using shift registers and XOR gates (modulo-2 adders).

Feedback Shift Register Encoder for $g(x) = x^3 + x + 1$



3. Convolutional Codes

Linear block codes process data block-by-block. **Convolutional codes**, in contrast, process information continuously, using internal shift-register states to create memory dependencies.

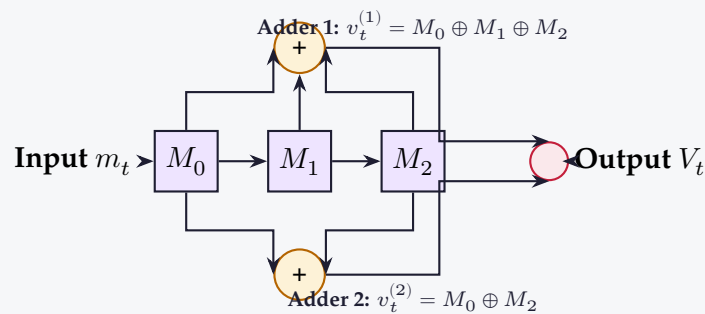
3.1 Basic Principles and Structure

A convolutional encoder is defined by three parameters (n, k, K) :

- n : Number of output bits produced in each shift cycle.
- k : Number of input bits processed in each shift cycle.
- K : Constraint length (number of shift register stages that affect the output).

The code rate is defined as $r = k/n$. The outputs are computed by modulo-2 addition of the current input and the values stored in the shift registers.

Convolutional Encoder Circuit (Rate 1/2, Constraint Length $K = 3$)

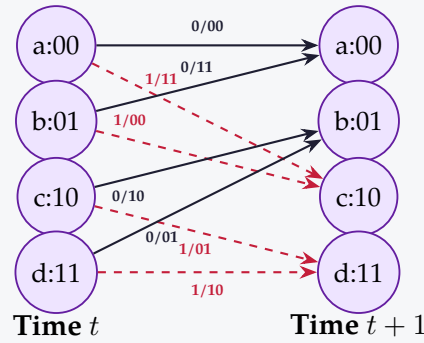


3.2 Representations of Convolutional Codes

Three graphical methods are used to describe a convolutional code:

1. **State Diagram:** Shows the states of the shift registers (excluding the current input) and the transitions between states caused by input '0' or '1'.
2. **Tree Diagram:** Shows all possible output paths as a branching tree structure, where the path branches upward for input '0' and downward for input '1'.
3. **Trellis Diagram:** A time-extended state transition diagram. It lists all possible states vertically at each time step, and shows transitions horizontally.

Trellis Diagram Transition Segment



3.3 Viterbi Decoding Algorithm

The Viterbi algorithm is the standard decoding method for convolutional codes. It finds the most likely sequence of transmitted symbols (minimum Hamming distance path) through the trellis.

- **Branch Metric:** The Hamming distance between the received bits and the expected output bits for a specific branch.
- **Path Metric:** The accumulated sum of branch metrics along a path.
- **Survivor Path:** At each node in the trellis, the incoming path with the lowest path metric is saved, and all other paths are discarded. This prevents exponential path growth.

4. Arithmetic Coding

Huffman coding assigns an integer number of bits to each symbol. If a symbol has a probability of 0.9, its ideal code length is $-\log_2 0.9 \approx 0.15$ bits, but Huffman must assign at least 1 bit, reducing efficiency. **Arithmetic coding** overcomes this constraint.

4.1 Core Concept

Arithmetic Coding: A non-block source coding algorithm that maps an entire sequence of symbols to a single fractional number in the interval $[0, 1)$. Symbols are represented by ranges proportional to their probabilities.

4.2 Encoding Procedure

Step 1: Initialize the interval boundaries: Low = 0, High = 1.

Step 2: For each symbol in the message:

$$\text{Range} = \text{High} - \text{Low} \quad (4)$$

$$\text{High} = \text{Low} + \text{Range} \times \text{Upper_Cum_Prob}(s_i) \quad (5)$$

$$\text{Low} = \text{Low} + \text{Range} \times \text{Lower_Cum_Prob}(s_i) \quad (6)$$

Step 3: Repeat for all symbols in the sequence.

Step 4: The final output is any floating-point number within the final [Low, High) interval.

Worked Example: Arithmetic Coding of "WET"

Problem: A source has three symbols $\{E, T, W\}$ with probabilities $\{0.5, 0.3, 0.2\}$.

- Calculate the cumulative probability ranges for the symbols.
- Encode the sequence "WET".

Solution:

(a) Probability Ranges:

Sort alphabet (say alphabetically $\{E, T, W\}$):

- Symbol E (prob = 0.5): Cumulative Interval $[0.0, 0.5)$
- Symbol T (prob = 0.3): Cumulative Interval $[0.5, 0.8)$
- Symbol W (prob = 0.2): Cumulative Interval $[0.8, 1.0)$

(b) Encode "WET":

- **Initial State:** Low = 0, High = 1, Range = 1.0.

- **First Symbol: 'W'** (Sub-interval $[0.8, 1.0)$):

$$\text{Low} = 0 + 1.0 \times 0.8 = 0.8$$

$$\text{High} = 0 + 1.0 \times 1.0 = 1.0$$

$$\text{Range} = 1.0 - 0.8 = 0.2$$

- **Second Symbol: 'E'** (Sub-interval $[0.0, 0.5)$ relative to current range):

$$\text{Low} = 0.8 + 0.2 \times 0.0 = 0.8$$

$$\text{High} = 0.8 + 0.2 \times 0.5 = 0.9$$

$$\text{Range} = 0.9 - 0.8 = 0.1$$

- **Third Symbol: 'T'** (Sub-interval $[0.5, 0.8)$ relative to current range):

$$\text{Low} = 0.8 + 0.1 \times 0.5 = 0.85$$

$$\text{High} = 0.8 + 0.1 \times 0.8 = 0.88$$

$$\text{Range} = 0.88 - 0.85 = 0.03$$

The final interval is $[0.85, 0.88)$. Any number in this range (e.g., **0.86**) represents the sequence "WET".

Quick Revision — Module III

Source & Channel Coding | PE-EC603D

Key Formulas at a Glance

Average Codeword Length	$L = \sum P_i l_i$ (bits/symbol)
Huffman Efficiency	$\eta = \frac{H(X)}{L} \times 100\%$
Cyclic Shift Shift	$C^{(1)} = (c_{n-1}, c_0, \dots, c_{n-2})$
Codeword Polynomial	$c(x) = m(x)g(x)$ (Non-systematic)
Systematic Codeword	$c(x) = x^{n-k}m(x) + r(x)$ where $r(x) = \text{rem} \left[\frac{x^{n-k}m(x)}{g(x)} \right]$
Syndrome Polynomial	$s(x) = \text{rem} \left[\frac{r(x)}{g(x)} \right]$ (If $s(x) = 0$, no error)
Convolutional Code Rate	$r = k/n$
Arithmetic Coding Range	Range = High – Low

One-Line Definitions

- **Huffman Coding:** A bottom-up optimal variable-length source coding technique based on sorting and merging probabilities.
- **Cyclic Code:** A linear block code where any cyclic shift of a codeword is also a valid codeword.
- **Generator Polynomial:** A unique polynomial $g(x)$ of degree $n - k$ that generates all codewords of a cyclic code.
- **Syndrome:** A polynomial remainder computed by the decoder to detect and locate errors in cyclic codes.
- **Constraint Length (K):** The number of shift register stages affecting output bits in a convolutional encoder.
- **Trellis Diagram:** A time-indexed state diagram representing paths and state transitions in convolutional codes.
- **Viterbi Decoder:** A maximum likelihood decoding algorithm using trellis paths to find the survivor path.
- **Arithmetic Coding:** A non-block coding method mapping a sequence of symbols to a single fractional number in $[0, 1)$.

Frequently Asked / Viva Questions

Q1. Why are Huffman codes uniquely decodable?

They possess the prefix-free property, meaning no codeword is a prefix of any

Frequently Asked / Viva Questions (contd.)

other. This allows symbol-by-symbol decoding without ambiguity.

Q2. What is the difference between systematic and non-systematic codes?

In systematic codes, the message bits appear unchanged at the beginning (or end) of the codeword. In non-systematic codes, message and parity bits are mixed.

Q3. State the algebraic condition for a polynomial to be a generator polynomial of a cyclic code.

It must be of degree $n - k$ and must be a factor of $x^n + 1$ over Galois Field $GF(2)$.

Q4. How are parity bits calculated in systematic cyclic codes?

By dividing $x^{n-k}m(x)$ by $g(x)$ using modulo-2 division. The remainder of this division is the parity check polynomial $r(x)$.

Q5. What hardware components are required to build a cyclic code encoder?

Linear Feedback Shift Registers (LFSRs) consisting of D flip-flops, XOR gates (modulo-2 adders), and switches.

Q6. State the difference between block codes and convolutional codes.

Block codes process data in fixed, independent blocks. Convolutional codes process data continuously, utilizing internal register states to create memory dependencies.

Q7. What parameters define a convolutional code?

(n, k, K) , where n is output bits per cycle, k is input bits per cycle, and K is the constraint length.

Q8. Explain the role of the Viterbi algorithm in convolutional decoding.

It acts as a maximum-likelihood decoder, finding the path through the trellis that minimizes the Hamming distance (or Euclidean distance) to the received sequence.

Q9. What is a "survivor path" in Viterbi decoding?

At each trellis node, two paths enter. The path with the lower accumulated metric (errors) is kept as the survivor path, and the other is discarded.

Q10. Why does arithmetic coding outperform Huffman coding for highly skewed probability sources?

Huffman requires an integer number of bits per symbol, causing inefficiency for symbols with probabilities > 0.5 . Arithmetic coding represents sequences as a single fraction.

Q11. How does the receiver decode an arithmetic code?

The receiver uses the received fractional value to determine which symbol's sub-interval contains the value, outputs the symbol, updates the interval, and repeats.

Frequently Asked / Viva Questions (contd.)

Q12. What is the relationship between the degree of $g(x)$ and cyclic code parameters?

The degree of the generator polynomial $g(x)$ is always equal to $n - k$, which is the number of parity check bits in the (n, k) code.

Comparison of Coding Techniques

Parameter	Cyclic Codes	Convolutional Codes
Structure	Block-based (processes blocks of length k).	Continuous (processes streams of bits).
Memory	Memoryless between blocks.	Memory-based (states of shift registers).
Implementation	LFSR shift registers and XOR gates.	Shift registers and modulo-2 adders.
Decoding	Syndrome calculations.	Trellis-based Viterbi decoding.
Applications	Error detection (CRC in Ethernet, USB).	Deep space communications, cellular (LTE).